ggfunction: Functions and Distributions in ggplot2

by Dusty Turner, David Kahle, Rodney X. Sturdivant, and James Otto

Abstract The ggfunction package provides ggplot2 layers for plotting mathematical functions, probability distributions, empirical summaries, and censored-data summaries directly from function definitions or samples. Its design has three parts. First, a dimensional taxonomy covers scalar functions, parametric curves, scalar fields, and vector fields. Second, probability layers display density, mass, distribution, survival, quantile, hazard, and cumulative-hazard functions with support-aware shading and documented conversion routes among representations. Third, empirical layers compute ECDFs, empirical quantiles, PP/QQ/SP diagnostics, empirical mass functions, empirical cumulative hazards, Kaplan–Meier survival curves, and Nelson–Aalen cumulative hazards. The displays are standard ggplot2 stats and geoms, so they compose with scales, coordinates, themes, facets, and annotations. Confidence displays are documented per layer, and fitted-null diagnostic bands are exploratory rather than calibrated unless separately adjusted.

1 Introduction

Many statistical graphics begin with a function rather than a data frame. An instructor shades the upper tail of a t distribution, a student compares an empirical CDF with a fitted model, or an analyst checks whether a censored sample is consistent with an exponential survival curve. In base R (R Core Team, 2026), these tasks are straightforward but often require hand-built grids, numerical integration, and ad hoc annotations. In **ggplot2** (Wickham, 2010, 2016), the built-in `stat_function()` covers only the univariate case $f : \mathbb{R} \to \mathbb{R}$ and leaves probability-specific calculations to the user.

ggfunction addresses this gap by making functions layer-level objects in **ggplot2**. Users supply an R function, optional parameters in `args`, and a domain. The layer evaluates the function, performs any requested statistical transformation, and returns ordinary **ggplot2** data for rendering. Figure 1 illustrates the motivating case: the user asks for a distributional tail probability, and the layer handles the quantile lookup, support-aware shading, and **ggplot2** display.

Two design choices distinguish **ggfunction** from small plotting wrappers. The probability layers share documented conversion routes among density or mass, distribution, survival, quantile, hazard, and cumulative-hazard representations, so one supplied characterization can drive several views of the same distribution. The package also separates statistical support from the display window: support governs normalization, integration, inversion, and probability shading, while `xlim` and coordinates govern what is visible. Together, theoretical curves, empirical summaries, censored-data estimators, and diagnostic plots use the same function-as-layer pattern while remaining ordinary **ggplot2** layers. The result is a practical, composable interface for common function- and distribution-based graphics in **ggplot2**.

The package is available from CRAN and can be installed with `install.packages("ggfunction")`. Development versions and issue tracking are available from `https://github.com/dusty-turner/ggfunction`. The package depends on **ggplot2**; bivariate highest-density regions are delegated to **ggdensity** (Otto and Kahle, 2023), and vector-field and stream-field layers delegate to **ggvfields** (Turner et al., 2026).

The rest of the article is organized around package design and workflows rather than a complete reference manual; full function-level documentation is available on the package website and in the vignettes. Section 2 describes the layer design and dimensional taxonomy. Section 3 presents three core workflows: mathematical functions, probability distributions, and empirical or censored data summaries. Section 4 summarizes implementation, numerical behavior, validation, and performance. Sections 5–7 discuss composability, related work, limitations, and future directions.

2 Design

2.1 Functions as layer-level objects

ggfunction does not change the grammar of graphics itself. It supplies new `ggplot2::layer()` constructors. Each constructor combines a `Stat` that evaluates a function or computes an empirical summary with a `Geom` that renders the returned data. This follows the **ggplot2** extension model: the

Figure 1: A motivating example: support-aware normal-density plotting and shading with a simple syntax. The shaded region is computed from the distribution, not by renormalizing over the displayed x range.

Table 1: Dimensional taxonomy for mathematical-function layers.

Signature	Layer	Display
$f : \mathbb{R} \to \mathbb{R}$	`geom_function_1d_1d()`	curve, optionally shaded
$\gamma : \mathbb{R} \to \mathbb{R}^2$	`geom_function_1d_2d()`	parametric path with parameter colour
$f : \mathbb{R}^2 \to \mathbb{R}$	`geom_function_2d_1d()`	raster, contours, or filled contours
$\mathbf{F} : \mathbb{R}^2 \to \mathbb{R}^2$	`geom_function_2d_2d()`	arrows or streamlines

result is a standard layer that can be added with +, trained by ordinary scales, clipped by coordinates, faceted, themed, and annotated.

The key interface choice is that functions are layer parameters, not aesthetics. This is deliberate. A function such as `dnorm()` or a bivariate density is not a column in the plot data nor is it a visual property of a geometric object; it is the object the stat evaluates over other aesthetics (typically the spatial ones `x` and `y`) to create plot data. Additional function parameters are supplied through `args`, mirroring `ggplot2::stat_function()` and avoiding anonymous wrappers. The basic specification syntax looks like this; the resulting plot is in Figure 1:

```
library("ggplot2"); theme_set(theme_minimal())
theme_update(panel.grid.minor = element_blank())
library("ggfunction")

ggplot() +
  geom_pdf(
    fun = dnorm, xlim = c(-3.5, 3.5),
    support = c(-Inf, Inf), p_lower = 0.025, p_upper = 0.975
  )
```

This design keeps the common case short while preserving explicit control over the domain, grid resolution, and rendering choices.

2.2 Dimensional taxonomy

The mathematical-function family is organized by domain and codomain dimension. Table 1 gives the taxonomy, and Figure 2 shows the corresponding displays. The names encode the function signature. That convention is more verbose than `geom_function()`, but it makes the dimensional assumptions visible at the call site. Probability and empirical layers use statistical names instead: `geom_pdf()`, `geom_ecdf()`, `geom_ecdf_km()`, and so on.

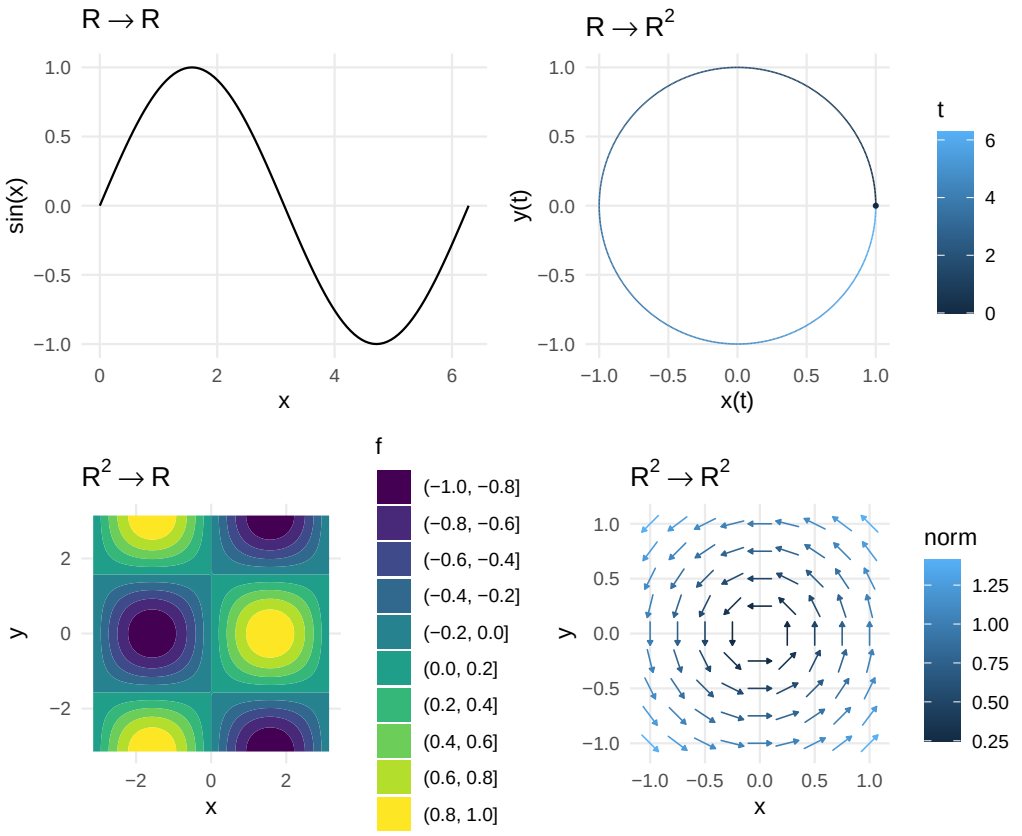

Figure 2: The four mathematical-function signatures supported by the ggfunction dimensional taxonomy: scalar functions, parametric curves, scalar fields, and vector fields.

3 Three workflows

3.1 Mathematical functions by dimension

For mathematical functions, the user's main decisions are the signature, the domain, and the visual encoding. A scalar function is evaluated on an equally spaced sequence over `xlim`. A parametric curve is evaluated over `tlim`; the returned length-two vector supplies the path coordinates. A scalar field is evaluated on an $n_x \times n_y$ grid and then handed to the corresponding **ggplot2** raster or contour machinery. A vector field is evaluated at seed points and then drawn by **ggvfields**; stream fields also delegate integration settings such as `method`, `max_it`, `T`, and `L`.

This workflow is useful when the data of interest are generated by a known mathematical rule. It also gives probability and diagnostic layers a common implementation pattern: create the grid, evaluate a function or summary, and return ordinary plot data.

The taxonomy is intentionally small. It covers the function signatures that can be displayed naturally in a two-dimensional plotting window without a separate three-dimensional graphics system. A scalar field, for example, can be displayed by mapping its output to colour or contour level while retaining the two input coordinates as the horizontal and vertical axes. A vector field can be displayed by drawing short vectors or streamlines at selected seed points. Higher-dimensional functions require additional choices–conditioning, projection, slicing, animation, or interactivity–that fall outside this package's current scope.

Two implementation details matter in this workflow. First, evaluation is grid-based. Increasing `n` improves smoothness and contour resolution, but it also increases computation, especially for two-dimensional grids. Second, `args` is spliced into the user's function at evaluation time. This lets a single function represent a parametric family without redefining closures, for example:

```
lissajous <- function(t, a = 3, b = 2, delta = pi / 2) c(sin(a*t + delta), sin(b*t))

ggplot() +
  geom_function_1d_2d(fun = lissajous, tlim = c(0, 2 * pi), args = list(a = 5, b = 4))
```

Table 2: Continuous distribution-function inputs and conversion routes. Native inputs are used directly; derived routes may require numerical integration, finite differences, interpolation, or root finding.

Target	Other sources	Route
PDF f	F, S, Q, or h	finite differences, interpolation, or hS
CDF F	f, S, Q, or h	`stats::integrate()` or identity
survival S	F, f, Q, or h	$1 - F$
quantile Q	F, f, S, or h	bracketed `stats::uniroot()` on F
hazard h	f, F, S, or Q	f/S with tail-stable S; $-(\log S)'$ from S; $f + F$ accepted
cum. hazard H	h, F, S, f, or Q	$-\log S$ or `stats::integrate()` on h

For scalar and vector fields, the same convention means that model parameters, physical constants, or tuning values can remain explicit in the plotting call. This is especially helpful in teaching settings, where the relationship between parameters and shape is often the point of the graphic. It also helps avoid subtle reproducibility problems: the plot source records both the mathematical function and the parameter values used to evaluate it.

Vector fields can be rendered either as local arrows or as streamlines. Both views start from the same vector-valued function, but they answer different visual questions. Arrows emphasize local direction and magnitude at selected grid points; streamlines emphasize integral curves of the field. In **ggfunction**, both displays are delegated to **ggvfields**.

3.2 Probability distributions

The probability workflow starts from one characterization of a distribution and asks for another visual representation. For example, users may have a density but want a CDF, a CDF but want a quantile function, or a hazard function but want a density. Native inputs are preferred when available as `fun`. When a native input is absent, **ggfunction** can derive the needed function through the routes shown in Tables 2 and 3; the input argument name signals the function type, e.g. supplying `hf_fun` to `geom_pdf()`.

The continuous one-dimensional layers use the standard identities among distribution functions. If f is a density and F is the CDF, then

$$F(x) = \int_{-\infty}^{x} f(t)\,dt, \qquad S(x) = 1 - F(x), \qquad Q(p) = F^{-1}(p).$$

For survival and reliability displays, the hazard and cumulative hazard are

$$h(x) = \frac{f(x)}{S(x)}, \qquad H(x) = -\log S(x), \qquad S(x) = \exp\{-H(x)\}.$$

The package does not rederive these relationships for the reader each time a geom is used. Instead, it encodes them in conversion helpers that are shared by the distribution layers. This keeps the user-facing API small while making the route explicit in documentation and validation tests.

Discrete distributions use analogous operations on support points. A PMF is accumulated by summation rather than integration, and a quantile function is a generalized inverse over the ordered support. Because discrete supports can be finite, infinite, integer-valued, or explicitly supplied, the `support` argument is more than a plotting convenience. Supplied support values are sorted before PMF values are accumulated; the vector is not treated as a traversal order. For example, if the mass at 2 is $1/2$, the mass at 1 is $1/3$, and the mass at 3 is $1/6$, then `support = c(2, 1, 3)` still yields CDF jumps at 1, then 2, then 3. The CDF jumps from 0 to $1/3$ at 1, to $1/3 + 1/2$ at 2, and to 1 at 3. This sorted support determines where step functions jump and how probability shading is resolved. For discrete cumulative targets that cannot be hit exactly, the discrete layers—`geom_pmf()` and the step geoms `geom_cdf_discrete()`, `geom_survival_discrete()`, and `geom_qf_discrete()`—include the first support point whose cumulative mass reaches or exceeds the target; discrete HDR shading similarly uses the smallest containing set, including all support points tied at the cutoff mass.

The bivariate probability layers have a narrower scope. `geom_pdf_2d()` draws theoretical bivariate densities as HDRs or rasters, and `geom_pmf_2d()` draws bivariate mass functions on a lattice. The package does not estimate bivariate densities from samples; that task belongs to **ggdensity**, which follows the same general design philosophy. It also does not provide arbitrary three-dimensional rendering of density surfaces. The goal is to display probability content in the same two-dimensional grammar used by the rest of **ggplot2**.

Table 3: Discrete distribution-function inputs and conversion routes. Computations are performed over the declared support.

Target	Other sources	Route
PMF	CDF or survival	support differences
CDF	PMF or survival	cumulative sums over sorted support
survival	PMF or CDF	$1 - F$ on sorted support
quantile	PMF, CDF, or survival	generalized inverse over sorted support

A single source can therefore drive several views of the same distribution. Figure 3 starts from the gamma density and asks **ggfunction** to draw related functions. Two of the panels use native functions, while the CDF and survival panels demonstrate PDF-derived routes. In a manuscript or analysis script, this makes the distributional relationship explicit in the plotting code. The design makes the code straightforward:

```
gamma_args <- list(shape = 2, scale = 1.5)
Sx <- c(0, Inf)
gx <- c(0, 10)
hx <- c(0.01, 10)

gamma_pdf <- ggplot() +
  geom_pdf(fun = dgamma, args = gamma_args, xlim = gx, support = Sx) +
  labs(title = "PDF", x = "x", y = "f(x)")

gamma_cdf <- ggplot() +
  geom_cdf(pdf_fun = dgamma, args = gamma_args, xlim = gx, support = Sx) +
  labs(title = "CDF from PDF", x = "x", y = "F(x)")

gamma_surv <- ggplot() +
  geom_survival(pdf_fun = dgamma, args = gamma_args, xlim = gx, support = Sx) +
  labs(title = "Survival from PDF", x = "x", y = "S(x)")

gamma_hazard <- ggplot() +
  geom_hf(pdf_fun = dgamma, cdf_fun = pgamma,
          args = gamma_args, xlim = hx, support = Sx) +
  labs(title = "Hazard from PDF + CDF", x = "x", y = "h(x)")

library("patchwork")
(gamma_pdf + gamma_cdf) / (gamma_surv + gamma_hazard)
```

The distinction between support and display window is important. The `xlim` argument controls what is drawn. The `support` argument controls distributional calculations: normalization checks, PDF-to-CDF integration, CDF-to-quantile inversion, and probability shading. For continuous one-dimensional probability layers, omitting `support` leaves the computational support at its default `c(-Inf, Inf)`; an `xlim` value alone does not redefine the distribution. Thus a 97.5% normal upper-tail request is interpreted with respect to the full normal distribution even if the visible window is narrower. Figure 4 shows lower-tail, central-interval, and two-tail requests.

This separation is easy to miss because ordinary plotting code often treats the x-axis limits as the whole domain. For probability displays, that would change the meaning of shaded area. Suppose a density has support $(-\infty, \infty)$ and the user displays only $[-2, 2]$. If a lower-tail request with $p = 0.95$ were normalized over the visible window, the shaded boundary would mark 95% of the mass inside $[-2, 2]$, not the 95th percentile of the distribution. That is rarely what a statistical diagram intends. In **ggfunction**, quantiles, tail probabilities, and central intervals are computed on the declared support; the coordinate system then determines which part of the result is visible.

The same distinction matters for bounded supports. A beta density should be integrated over $[0, 1]$, not over the displayed portion of that interval. A Weibull or exponential hazard should accumulate from time zero unless the user specifies another origin. A custom distribution with support on a nonstandard finite interval should declare that interval so numerical integration and root-finding do not waste effort outside the distribution's domain or return misleading boundary behavior. The `support` argument is therefore a statistical parameter of the layer, not a cosmetic axis setting.

Discrete layers have a different fallback because their computation requires a finite set of support points. If `support` is omitted and `xlim` is supplied, the package evaluates the integer support from

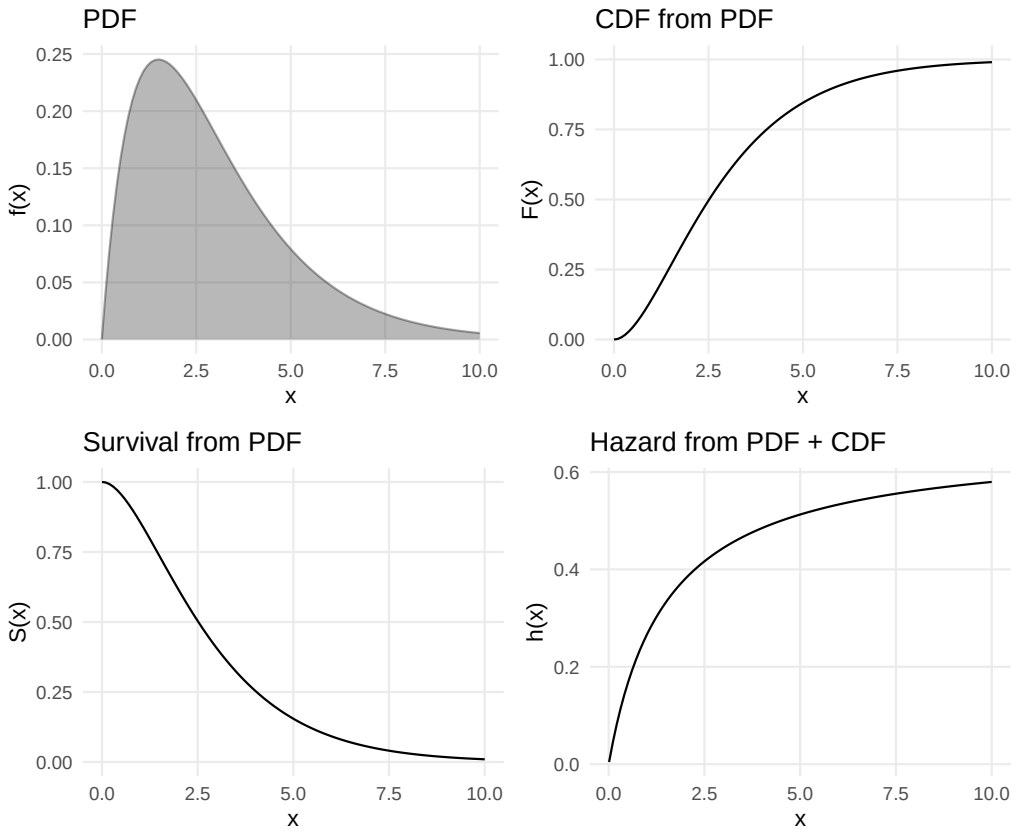

Figure 3: One gamma distribution viewed through four related functions. The PDF panel uses a native density; the CDF and survival panels derive from the density; the hazard panel uses the documented PDF + CDF bundle.

`ceiling(xlim[1])` through `floor(xlim[2])`; if both are omitted, it uses a small default support. This is convenient for quick binomial and Poisson examples, but it also means that `xlim` is part of the discrete computation unless `support` is supplied. The package does not issue a periodic startup-style message for this documented default; in analyses where `xlim` is only a visual crop, users should declare the full `support` explicitly and crop with coordinates.

Display windows still have an important role. They let the user focus on the region where a comparison is readable, crop extreme tails, or align several plots with a common x-axis. The recommended pattern is to compute on the distributional support and use the plotting window only for visual emphasis. For HDR displays, the analogous pattern is to compute over `hdr_xlim` and `hdr_ylim` and use `coord_cartesian()` if a narrower view is desired. This is the same principle in one or two dimensions: probability content belongs to the computation domain, while readability belongs to the display domain.

Highest density regions (HDRs) are threshold-based regions of largest density containing a requested probability. In one dimension, **ggfunction** computes an approximate threshold on a grid over `hdr_xlim`, using `xlim` only as the fallback HDR computation interval when `hdr_xlim` is omitted. In two dimensions, `geom_pdf_2d()` adapts the package's vector-argument density interface to **ggdensity**'s `geom_hdr_fun()` and `geom_hdr_lines_fun()` interfaces. **ggdensity** uses its own `xlim/ylim` arguments as the functional HDR domain; **ggfunction** exposes `hdr_xlim` and `hdr_ylim` so users can choose a larger HDR computation domain while keeping the displayed `xlim/ylim` narrow. These HDRs are gridded approximations whose accuracy depends on the computational domain and grid resolution. For a narrower view after computing on a wider domain, users should compute over the larger `hdr_xlim/hdr_ylim` and use `coord_cartesian()` for display.

The bivariate PMF layer uses the same vector-argument convention as the bivariate PDF layer, but evaluates on a lattice. Figure 6 shows the same independent product-binomial mass function in point and tile modes. Point mode maps probability to area, while tile mode maps probability to fill.

Discrete distributions follow the same ideas on a finite or integer support. The PMF layer uses lollipops, discrete CDF/quantile/survival layers use step functions, and discrete HDRs include all support points tied at an HDR cutoff. Figure 7 uses a deliberately nonconsecutive support to show why the support itself is part of the statistical object: cumulative sums and inverse lookups must be

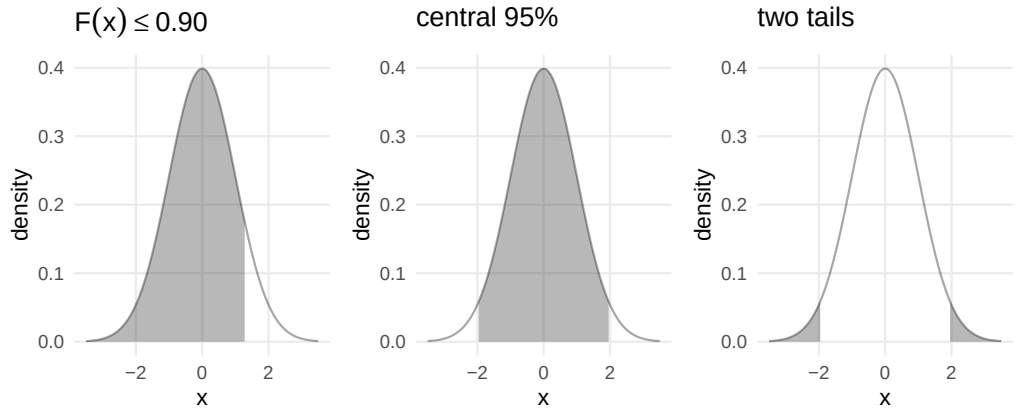

Figure 4: Three probability-shading requests for the standard normal distribution: a lower-tail probability, a central interval, and two tails outside a central interval.

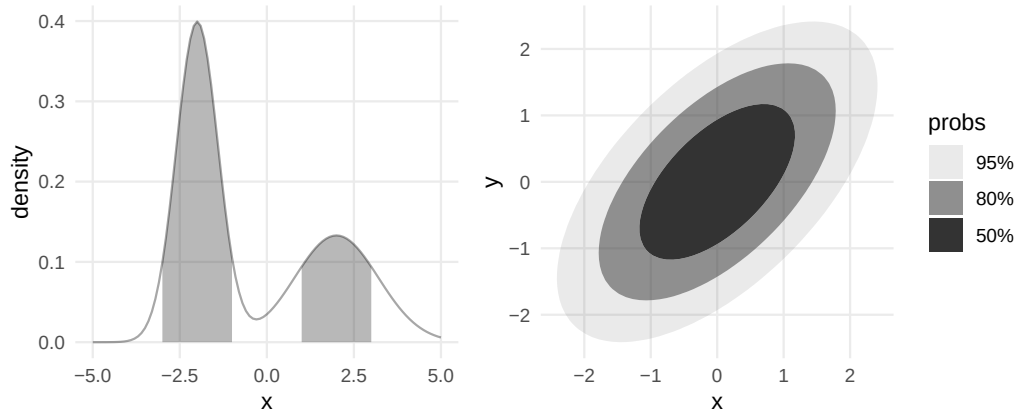

Figure 5: Approximate highest density regions. Left: a disconnected 80% grid-based HDR for a bimodal univariate density. Right: bivariate normal HDRs delegated to ggdensity.

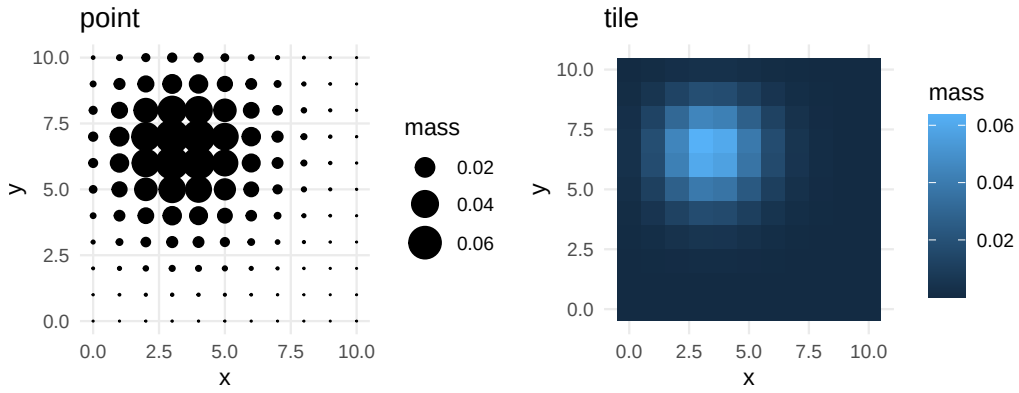

Figure 6: A bivariate probability mass function displayed as a balloon plot and as a tile heatmap.

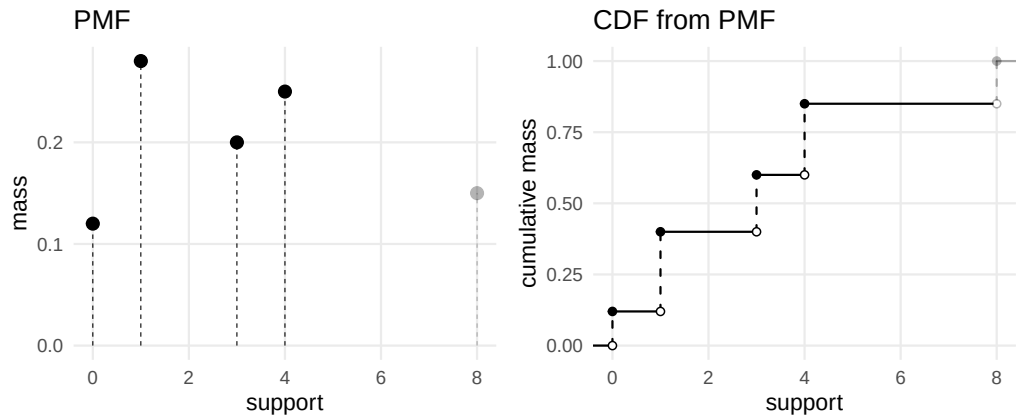

Figure 7: A discrete distribution on a nonconsecutive support. Both layers resolve the same lower cumulative shading request; the discrete CDF is derived from the same PMF by cumulative summation over the declared support.

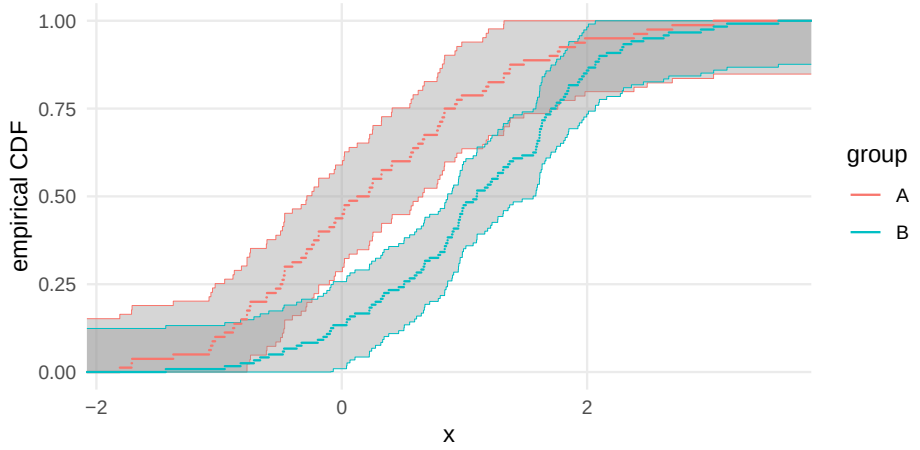

Figure 8: Grouped ECDFs with DKW/Massart confidence bands. Each group receives a separate band computed from its own sample size.

performed on the declared support, not on all integers between the minimum and maximum. For examples involving ties and bivariate mass functions, see the probability and numerical-conversion vignettes.

3.3 Empirical distributions and censored summaries

When users have data, the empirical workflow compares sample summaries with distributional structure. For complete samples, `geom_ecdf()` and `geom_eqf()` draw empirical distribution and quantile functions. Their confidence bands use the Dvoretzky–Kiefer–Wolfowitz inequality with Massart's sharp constant (Massart, 1990). For a fully specified distribution F, the ECDF band

$$\hat{F}_n(x) \pm \sqrt{\frac{\log(2/\alpha)}{2n}}$$

has simultaneous coverage at least $1-\alpha$, clipped to $[0,1]$. Grouped ECDFs receive group-specific bands, as in Figure 8. The empirical quantile band is obtained by inverting these CDF limits. This inversion is visually useful but produces wide or partially uninformative tail regions when $p \pm \varepsilon$ reaches 0 or 1, which is a property of the DKW bound rather than a plotting artifact.

The band drawn in Figure 8 is the direct finite-sample DKW/Massart band. The package vignette `coverage-simulation` gives additional simulation checks across distributions, sample sizes, and replicate counts.

The same complete-data sample can be viewed on the cumulative-hazard scale. This is useful when the theoretical hazard has a simpler visual form than the CDF. For exponential data, the cumulative hazard is linear, so departures from linearity are easier to see after transformation. The complete-

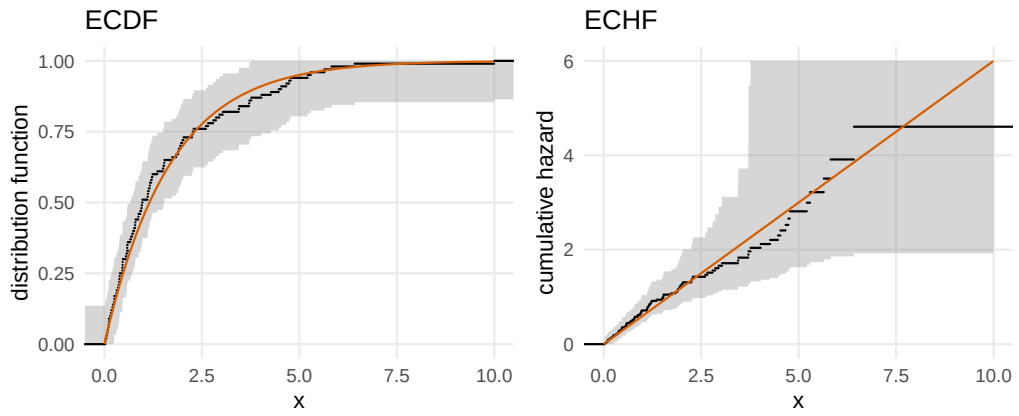

Figure 9: The same complete exponential sample shown as an ECDF and as an empirical cumulative hazard. The ECDF is curved under an exponential model, while the cumulative-hazard target is linear. The displayed ECHF upper band is clipped where the transformed upper DKW bound saturates.

data `geom_echf()` layer applies the monotone transform $H(x) = -\log\{1 - F(x)\}$ to the ECDF and to the DKW limits. Where the upper DKW limit reaches 1, the transformed upper limit is infinite; the figure clips that edge for display. Figure 9 shows the same sample on the ECDF and empirical cumulative-hazard scales.

Diagnostic layers use the same probability-scale argument. `geom_ppplot()` compares theoretical and empirical probabilities, `geom_qqplot()` compares theoretical and sample quantiles, and `geom_spplot()` applies Michael's arcsine-square-root stabilization (Michael, 1983) to the PP plot. Observations enter through the non-positional `sample` aesthetic, as in `ggplot2::geom_qq()`: **ggplot2** transforms position aesthetics before a layer's statistic runs, so a positional input would hand the null distribution transformed values whenever an axis carries a transformation, whereas `sample` guarantees that the null CDF or quantile function always receives the raw data. Because the PP and SP bands are continuous-null procedures, requesting one requires declaring `null_type = "continuous"`. When the null distribution is fully specified via an input function `fun`, the DKW/Massart band is a finite-sample simultaneous band for the probability integral transform. When parameters are estimated from the same data being plotted, that argument no longer gives the stated finite-sample simultaneous calibration. This distinction is important in practice. In teaching, it is common to compare a sample with `pnorm` after estimating mean and standard deviation from the same data. **ggfunction** will draw the requested visual band, but it does not perform a Lilliefors (Lilliefors, 1967) or other composite-null adjustment. The plotted band can still be a useful exploratory diagnostic; users who need a calibrated composite-null goodness-of-fit procedure should use a method designed for that setting.

For right-censored data, `geom_ecdf_km()` computes the Kaplan–Meier product-limit estimator (Kaplan and Meier, 1958) from observed times and an event indicator. It uses risk sets and displays censoring marks by default. Its confidence band uses Greenwood's variance estimator (Greenwood, 1926) and the equal-precision construction of Nair (1984), which is simultaneous and asymptotic. The companion `geom_echf_na()` layer computes the Nelson–Aalen cumulative hazard estimator (Nelson, 1972; Aalen, 1978) and uses the implemented pointwise normal confidence intervals. Figure 11 shows the two estimators side by side. For censored observations, ECDF-based complete-data summaries are no longer appropriate; the risk-set calculations in the Kaplan–Meier and Nelson–Aalen estimators are essential.

At each distinct event time t_j, let d_j denote the number of events and n_j the number at risk immediately before t_j. The Kaplan–Meier estimator displayed by `geom_ecdf_km()` is

$$\hat{S}(t) = \prod_{t_j \le t} \left(1 - \frac{d_j}{n_j}\right),$$

and the Nelson–Aalen estimator displayed by `geom_echf_na()` is

$$\hat{H}(t) = \sum_{t_j \le t} \frac{d_j}{n_j}.$$

The package computes both from the same internal risk-set table. This shared tabulation keeps event counts, censoring counts, risk sets, and variance estimates consistent between the two censored-data layers.

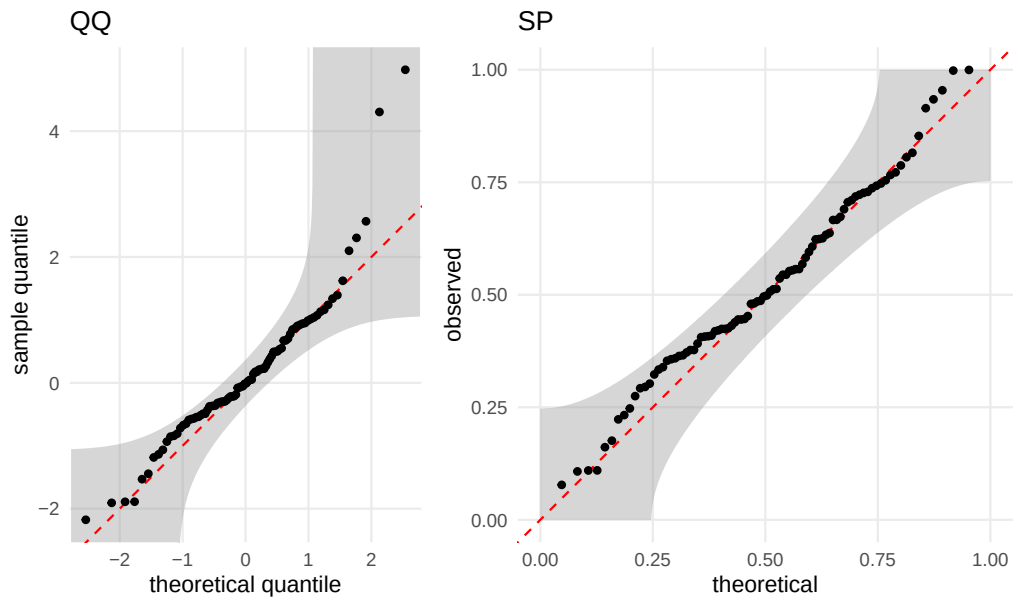

Figure 10: QQ and stabilized probability plots for one sample against a fully specified standard normal null. Fitted-null diagnostics use the same visual band but require separate calibration for formal testing.

The confidence displays intentionally differ. The Kaplan–Meier band uses Greenwood's variance estimate and an equal-precision critical value following Nair's construction. The displayed limits are additive on the survival scale, $\hat{S}(t) \pm c_{\mathrm{EP}} \mathrm{se}\{\hat{S}(t)\}$, and are clipped to $[0, 1]$. The critical value is computed from Greenwood-scale endpoints at the first and last valid event times, so the band is a simultaneous large-sample band over the displayed event-time range, not an exact finite-sample band. The Nelson–Aalen intervals use the Nelson variance estimate

$$\widehat{\mathrm{Var}}\{\hat{H}(t)\} = \sum_{t_j \le t} \frac{d_j}{n_j^2},$$

and a normal critical value at each time point. Its intervals are symmetric on the cumulative-hazard scale before the lower limit is clipped at zero. It is therefore pointwise, not simultaneous. The visual encodings intentionally differ: the simultaneous Kaplan–Meier band is drawn as a ribbon, while the pointwise Nelson–Aalen intervals are drawn as capped error bars at event times. The examples use this distinction deliberately.

4 Implementation, numerical behavior, and performance

4.1 ggplot2 stat-geom architecture

Each user-facing layer constructs one or more ggplot2 layers. The stat component validates inputs, evaluates functions or tabulates empirical summaries, and returns a data frame with computed variables such as `x`, `y`, `ymin`, `ymax`, or `probs`. The geom component draws those data using existing ggplot2 infrastructure when possible: paths and ribbons for curves and bands, rasters and contours for scalar fields, and step-like geoms for discrete and empirical functions.

The two main delegated components are explicit. Vector and stream fields are wrappers around ggvfields vector-field and stream-field layers. Bivariate PDF HDR displays are thin wrappers around ggdensity's functional HDR geoms; the raster mode for `geom_pdf_2d()` uses the scalar-field machinery instead. This delegation keeps **ggfunction** focused on function interfaces, probability semantics, and validation, while not duplicating other work maintained by the authors.

The layer constructors also provide a predictable data contract. Most users do not inspect the built data, but doing so is useful for tests, extensions, and debugging. Table 4 summarizes representative computed variables. These are not new aesthetics that users must supply; they are values created by the stat and then consumed by the geom.

This contract is also how the package is tested. Unit tests build plots with `ggplot_build()`, inspect the computed data frames, and compare band limits, support handling, route accuracy, and error

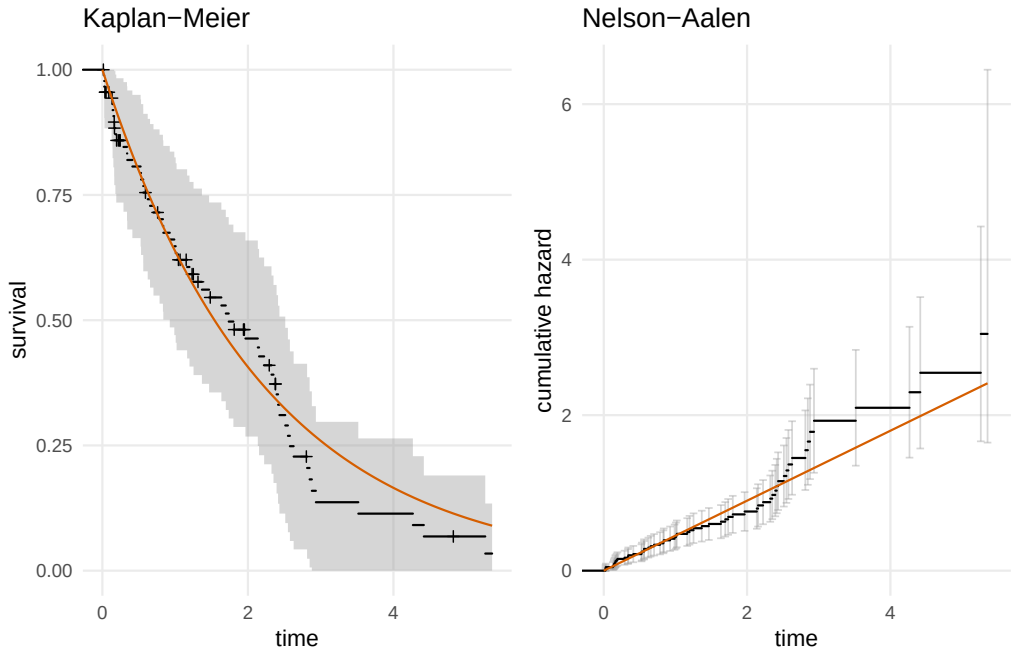

Figure 11: Censored-data summaries from the same simulated sample. The smooth orange curves show the generating Exp(0.45) survival and cumulative-hazard targets. Left: Kaplan–Meier survival with censor marks and a simultaneous Greenwood/Nair band. Right: Nelson–Aalen cumulative hazard with pointwise normal intervals.

Table 4: Representative computed variables returned by ggfunction stats.

Family	Variables	Geom
curves	x, y	path/area
fields	x, y, z	raster/contour
continuous prob.	x, y, shade vars	area/path/ribbon
discrete prob.	x, y, mass/HDR	lollipop/step
empirical	x, y, band limits	step/ribbon
censored	x, y, band/interval/censor	step/ribbon/bars/marks
diagnostics	x, y, order/probs	points/ribbon/line

Table 5: Representative numerical conversion accuracy from saved validation output. Errors are maximum absolute differences on the fixed grids described in the text.

Route	Distribution	Grid	Max error	Note
PDF → CDF	normal	401	5.82e-07	–
CDF → PDF	normal	401	1.29e-11	finite differences
CDF → QF	normal	199	3.49e-09	adaptive uniroot
HF → PDF	exponential	401	0.00e+00	support lower endpoint 0
HF → PDF	weibull	401	5.90e-08	support lower endpoint 0
PDF → CDF	beta(2,5)	401	3.33e-16	finite support
PMF → CDF	binomial(20,0.35)	21	5.55e-16	–
PMF → QF	binomial(20,0.35)	199	0.00e+00	–

messages with expected values. The tests therefore exercise the same data that a downstream extension would see.

4.2 Numerical conversions

The conversion code is intentionally conservative. Native functions are used directly. Derived functions are tagged internally as approximate, and their accuracy depends on smoothness, support, tail behavior, grid resolution, and whether the supplied function is numerically stable. The main conversion methods are:

- PDF to CDF: `stats::integrate()` over the declared support.
- CDF to PDF: central finite differences with support-aware endpoint handling, implemented directly rather than through an external numerical-differentiation dependency.
- CDF to quantile: bracketing and `stats::uniroot()`.
- Quantile to CDF: `stats::approxfun()` interpolation on a probability grid.
- Survival to CDF or cumulative hazard: algebraic transformations.
- Survival or PDF to hazard: the logarithmic derivative $-\frac{d}{dx}\log S(x)$, or f divided by an upper-tail integral of f; both remain stable deep in the tail, where forming $f/(1-F)$ loses all precision once F rounds to 1.
- Hazard to cumulative hazard: `stats::integrate()` from the hazard origin, with exact endpoint values ($H=\infty$, hence $F=1$ and $S=0$, at the upper support endpoint) rather than attempted divergent integrals.
- Hazard to PDF: $f(x) = h(x)\exp\{-H(x)\}$ after deriving H.
- Discrete conversions: cumulative sums over sorted support and generalized inverse lookup.

These routes are meant to support plotting, diagnostics, and teaching examples, not to replace distribution-specific numerical libraries. The numerical CDF derived from a smooth bounded density can be extremely accurate on a moderate grid. The same default settings may be inadequate for a density with sharp spikes near a boundary or for a heavy-tailed distribution whose visually interesting mass lies outside the displayed window. The API therefore exposes the most important controls–the declared support, the display window, the grid resolution, and the hazard integration origin–instead of hiding all numerical choices behind a single plotting call.

For hazards, the lower integration endpoint deserves special mention. If the support has a finite lower endpoint and `hf_lower` is left at its default of `-Inf`, the package uses the lower support endpoint. This makes common nonnegative lifetime distributions behave naturally when users write `support = c(0, Inf)`. If neither the support nor the hazard origin is finite, integrating a hazard can be ill-posed; the layer then warns or fails rather than silently drawing a misleading curve.

Tables 5 and 6 report representative local validation results from the accuracy and benchmark scripts in `inst/benchmarks/`. These scripts are manual validation aids rather than CRAN tests. They install the local package, compare fixed-grid function values, or time `ggplot_build()` for representative plots. The saved outputs used here are in `inst/benchmarks/results/` and record the run date, platform, R version, package versions, grid details, and benchmark iteration counts.

The reported run (2026-07-29 19:28:09 CDT) was on Darwin 25.5.0 arm64 using R 4.6.1 (2026-06-24), **ggfunction** 0.1.0, **ggplot2** 4.0.3.9000, **ggdensity** 1.0.1, and **ggvfields** 1.0.0. The benchmark script used **bench** (Hester and Vaughan, 2025) 1.1.4. Accuracy grids used 401 x-values for the normal, exponential, Weibull, and beta checks, 199 probabilities for quantile checks, and the 21-point binomial support.

The tables should be read as reproducibility checks, not universal guarantees. Heavy tails, discontinuities, bounded supports, singular hazards, and nearly flat CDF regions are harder cases. Users can

Table 6: Representative local performance checks from saved benchmark output. Timings are medians over 10 iterations of plot construction, not full device rendering.

Case	Domain	Median	Iterations
geom_pdf(fun = dnorm)	101 x values on [-4, 4]	17.5ms	10
geom_pdf(cdf_fun = pnorm)	101 x values on [-4, 4]	19.8ms	10
geom_cdf(pdf_fun = dnorm)	101 x values on [-4, 4]	22.2ms	10
geom_function_2d_1d()	80 x 80 grid on [-pi, pi]^2	64.7ms	10
geom_pdf_2d(), ggdensity HDR	60 x 60 grid on [-3, 3]^2	90.8ms	10

improve behavior by supplying the native distribution function, declaring finite `support`, increasing `n`, or setting a finite `hf_lower` for hazards whose origin is known.

4.3 Performance and scaling

For native one-dimensional functions, build time is dominated by evaluating the function at `n` grid points and by **ggplot2** layer construction, so the cost scales roughly linearly in `n`. Scalar fields and bivariate density displays evaluate an $n_x \times n_y$ grid, so their computational cost scales like n^2. Conversions involving integration or root-finding are more expensive than native evaluations because they call numerical routines at many grid points. Stream fields depend additionally on ODE settings and the behavior of the vector field; rendering time can dominate for dense contours, HDRs, and streamlines.

4.4 Validation and warnings

The package validates common failure modes before plotting. Densities are checked for approximate normalization over their declared support, PMFs are checked for non-negative finite mass and approximate summation, CDF-like inputs are checked for monotonicity where the route requires it, and support/window diagnostics warn when an HDR computation domain omits substantial probability mass. Computed survival, quantile, hazard, and cumulative-hazard curves are soft-checked as well: survival curves for unit-to-zero endpoint behavior and monotone decrease, quantile curves for monotone increase within the declared support, hazards for non-negativity, and cumulative hazards additionally for monotone increase. These diagnostics alert without aborting and can be disabled per layer via `check = FALSE` or globally via the package option. Invalid source combinations are rejected. Most distribution geoms require exactly one source function, but hazard layers also accept the documented bundle `pdf_fun` + `cdf_fun`, which avoids deriving either component numerically.

The validation policy aims to catch mistakes that are easy to make in plotting code. Examples include using the displayed window as if it were the full support, supplying an unnamed `args` list, drawing a density that integrates far from one, evaluating a CDF-derived PDF outside the support, or asking for a quantile route without finite bracketing information. When the package can continue safely, it warns and returns missing values for failed points. When the requested route is logically ambiguous, it errors before building the plot; for example, `geom_pdf(cdf_fun = pnorm, survival_fun = function(x) 1 - pnorm(x))` supplies two possible sources for the same PDF target and is rejected.

The checks are deliberately lightweight enough for interactive plotting. They do not prove that an arbitrary user-supplied function is a valid distribution function. In particular, they cannot rescue a discontinuous CDF from finite differencing artifacts, identify every singular hazard, or choose a universally appropriate HDR grid. The documentation and numerical-conversion vignette therefore emphasize native distribution functions and explicit support declarations whenever accuracy matters.

The package also includes supporting infrastructure for submission and maintenance. Documentation generated with **roxygen2** (Wickham et al., 2026) records the accepted source combinations, approximation routes, support/window distinction, and warning behavior for each user-facing layer. Vignettes move longer teaching examples and validation stories out of the article: probability geoms, empirical geoms, numerical conversions, survival analysis, and DKW coverage simulation each have a focused home. Manual benchmark scripts in `inst/benchmarks/` provide reproducible accuracy and timing checks without running during `R CMD check`. This separation keeps CRAN checks fast while preserving evidence for manuscript claims.

5 Composability and limitations

Because **ggfunction** returns standard **ggplot2** layers, its outputs compose with scales, coordinates, themes, facets, and annotations in ways **ggplot2** users have come to expect. A theoretical curve can be overlaid on an ECDF, a survival curve can be added to a Kaplan–Meier estimate, and a scalar field can be recolored with ordinary fill scales. This composability is the main practical benefit of implementing functions inside the **ggplot2** layer system.

Composability also makes the package useful in documents and teaching material. A probability layer can sit next to a simulation layer in a **patchwork** layout (Pedersen, 2025), use the same theme as the surrounding analysis, or inherit a coordinate system that zooms the display without changing the distributional calculation. The support/window distinction is especially important here: a plot can compute probability content over the full support while using `coord_cartesian()` to focus on the region where the visual comparison matters.

The same design also defines the limits. Functions are layer parameters, not aesthetics, so faceting over a data-frame column of function objects is not supported in the ordinary grammar-of-graphics sense. Numerical conversions are approximations and can fail for difficult distributions. HDRs are grid-based, so their thresholds depend on the computational domain and grid resolution. Two-dimensional grids can become expensive at high resolution. Censored-data confidence displays have the validity described in the implementation: Kaplan–Meier uses the implemented simultaneous EP Greenwood/Nair band, while Nelson–Aalen uses implemented pointwise normal intervals.

There are also statistical limits that no plotting interface can remove. Probability shading is only as meaningful as the distribution supplied to the layer. A diagnostic band for a fully specified null model has a different interpretation from the same band drawn after fitting parameters. A gridded HDR computed on too narrow a domain may have the wrong threshold. A survival curve with heavy censoring in the tail can have a visually wide confidence display because the risk set is small. The package tries to make these situations visible, but the user remains responsible for choosing a model and interpreting the plot in context.

6 Related work

ggfunction is closest to tools that bridge statistical or mathematical objects and graphics. The compact summaries below describe the closest **ggplot2** extensions and adjacent packages. Table 7 emphasizes function-layer design; Table 8 emphasizes empirical, diagnostic, and survival displays. The intent is complementarity rather than replacement: several packages solve different problems better than **ggfunction** does.

Table 7: Closest function-layer and field-display packages.

Package	Relationship
`ggplot2::stat_function()`	native univariate function-curve baseline
ggvfields	delegated vector-field and stream-field rendering
metR	stronger support for precomputed gridded fields
ggformula/**mosaicCalc**	formula-oriented front ends for calculus and teaching

Table 8: Adjacent distribution, diagnostic, and survival-display packages.

Package	Relationship
ggdist	broader distribution displays
ggdensity	bivariate HDR engine used here
hdrcde	HDR objects outside layer taxonomy
qqplotr	wider band/detrend menu
survminer/**ggsurvfit**	risk-table/test workflows

ggplot2 (Wickham, 2010, 2016) and Wilkinson's grammar (Wilkinson, 2005) provide the conceptual and software foundation. **ggdist** (Kay, 2024) is broader for uncertainty visualization. **ggdensity** (Otto and Kahle, 2023) supplies the bivariate HDR machinery used here. **ggvfields** (Turner et al., 2026) supplies vector-field and stream-field infrastructure. **metR** (Campitelli, 2024) and **hdrcde** (Hyndman et al., 2024) cover adjacent meteorological and HDR-focused use cases. **ggformula** (Kaplan and Pruim, 2026), **mosaic** (Pruim et al., 2017), and **mosaicCalc** (Kaplan et al., 2024) provide formula-oriented front ends for functions, distributions, calculus, and introductory statistics; **ggfunction** instead emphasizes

native stats/geoms, support-aware probability semantics, and conversion routes among distribution representations.

The package boundaries are therefore partly philosophical. **ggdist** starts from distributions and uncertainty summaries and provides a rich visual vocabulary for communicating posterior draws, intervals, slabs, and dots. **ggfunction** starts from ordinary R functions and asks how they should be evaluated inside a **ggplot2** layer. **ggdensity** focuses on bivariate density estimation and probability-calibrated contours; **ggfunction** uses that machinery when the user supplies a theoretical bivariate density. **ggvfields** focuses on vector-field displays and integration; **ggfunction** exposes those displays through the dimensional taxonomy so scalar and vector-valued functions share a naming scheme.

For diagnostic plots, **qqplotr** (Almeida et al., 2018, 2025) is the closest **ggplot2** extension, providing Q-Q and P-P points, reference lines, confidence-band constructions, and detrended displays. **ggfunction** overlaps through `geom_qqplot()` and `geom_ppplot()`, and adds `geom_spplot()` within the same support-aware probability framework used by its ECDF and quantile layers; users who need a wider diagnostic-band or detrending menu should use **qqplotr**. For right-censored data, **survminer** (Kassambara et al., 2026) and **ggsurvfit** (Sjoberg et al., 2025) provide publication-oriented time-to-event figures with survival-analysis furniture such as risk tables, censoring summaries, stratified comparisons, and ready-to-share themes; **ggsurvfit** is itself modular and **ggplot2**-compatible. The censored-data layers in **ggfunction** emphasize a lower-level role inside the same support-aware probability framework used by its other empirical geoms: `geom_ecdf_km()` and `geom_echf_na()` share one risk-set tabulation, use deliberately different confidence encodings for simultaneous and pointwise displays, and overlay theoretical survival or cumulative-hazard curves as ordinary layers. They do not aim to replace survival-workflow packages.

This scope keeps the package lightweight. It does not define distribution objects, fit models, estimate bivariate densities from samples, or implement a general symbolic mathematics system. Instead it provides a bridge from function definitions and empirical summaries to ordinary **ggplot2** layers. That bridge is useful precisely because it leaves the rest of the graphics system unchanged.

7 Discussion and future directions

While **ggfunction** is a large package, it makes a narrow commitment: take a mathematical or statistical function, evaluate it safely enough for plotting, and return a standard **ggplot2** layer. That commitment covers common teaching and diagnostic tasks without asking users to build grids, integrate densities, or hand-code confidence ribbons, and in doing so alleviates pain points experienced by countless **ggplot2** users in a thoughtful way.

Future work could add better support for collections of functions, richer interfaces for distribution objects, more user controls for numerical conversion tolerances, conditional distributions and HDRs, additional censored-data confidence displays, and higher-level wrappers for common teaching workflows. The benchmark and accuracy scripts added during package hardening provide a starting point for tracking these changes over time.

The most natural next step is support for function collections. Many classroom and applied examples compare a family of functions–normal densities with different scales, hazard functions under several parameter settings, or solution curves from different initial conditions. Today these can be overlaid by adding multiple layers manually. A future interface could make such collections more data-like while respecting the fact that functions are not ordinary aesthetics.

Another direction is more explicit numerical control. The defaults are chosen for readable interactive graphics, but some users will want tighter integration tolerances, adaptive grids, or route-specific diagnostics. Exposing those controls without turning simple plots into numerical-analysis exercises is a design challenge. The validation scripts introduced here give the package a baseline against which such changes can be checked.

8 Acknowledgments

James Otto contributed early drafts of the CDF and ECDF-related stats and geoms. Rodney Sturdivant contributed early design feedback. The authors used OpenAI's GPT 5.5 and Anthropic's Claude Opus 4.8 extensively as programming and drafting assistants during package development and manuscript revision; the authors reviewed and are responsible for the package code, examples, and manuscript claims.

References

O. Aalen. Nonparametric inference for a family of counting processes. *The Annals of Statistics*, 6(4): 701–726, 1978. doi: 10.1214/aos/1176344247. [p9]

A. Almeida, A. Loy, and H. Hofmann. ggplot2 compatible quantile-quantile plots in R. *The R Journal*, 10(2):248–261, 2018. doi: 10.32614/RJ-2018-051. [p15]

A. Almeida, A. Loy, and H. Hofmann. *qqplotr: Quantile-Quantile Plot Extensions for ggplot2*, 2025. URL https://CRAN.R-project.org/package=qqplotr. R package version 0.0.7. [p15]

E. Campitelli. *metR: Tools for Easier Analysis of Meteorological Fields*, 2024. URL https://CRAN.R-project.org/package=metR. R package. [p14]

M. Greenwood. *A Report on the Natural Duration of Cancer*. Number 33 in Reports on Public Health and Medical Subjects. His Majesty's Stationery Office, London, 1926. [p9]

J. Hester and D. Vaughan. *bench: High Precision Timing of R Expressions*, 2025. URL https://CRAN.R-project.org/package=bench. R package version 1.1.4. [p12]

R. J. Hyndman, J. Einbeck, and M. P. Wand. *hdrcde: Highest Density Regions and Conditional Density Estimation*, 2024. URL https://CRAN.R-project.org/package=hdrcde. R package. [p14]

D. Kaplan and R. Pruim. *ggformula: Formula Interface to the Grammar of Graphics*, 2026. URL https://CRAN.R-project.org/package=ggformula. R package version 1.0.1. [p14]

D. T. Kaplan, R. Pruim, and N. J. Horton. *mosaicCalc: R-Language Based Calculus Operations for Teaching*, 2024. URL https://CRAN.R-project.org/package=mosaicCalc. R package version 0.6.4. [p14]

E. L. Kaplan and P. Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958. doi: 10.1080/01621459.1958.10501452. [p9]

A. Kassambara, M. Kosinski, P. Biecek, and F. Scheipl. *survminer: Drawing Survival Curves using ggplot2*, 2026. URL https://CRAN.R-project.org/package=survminer. R package version 0.5.2. [p15]

M. Kay. ggdist: Visualizations of distributions and uncertainty in the grammar of graphics. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):983–993, 2024. doi: 10.1109/TVCG.2023.3327195. [p14]

H. W. Lilliefors. On the Kolmogorov-Smirnov test for normality with mean and variance unknown. *Journal of the American Statistical Association*, 62(318):399–402, 1967. doi: 10.1080/01621459.1967.10482916. [p9]

P. Massart. The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality. *The Annals of Probability*, 18(3):1269–1283, 1990. doi: 10.1214/aop/1176990746. [p8]

J. R. Michael. The stabilized probability plot. *Biometrika*, 70(1):11–17, 1983. doi: 10.1093/biomet/70.1.11. URL https://www.jstor.org/stable/2335939. [p9]

V. N. Nair. Confidence bands for survival functions with censored data: A comparative study. *Technometrics*, 26(3):265–275, 1984. doi: 10.1080/00401706.1984.10487964. [p9]

W. Nelson. Theory and applications of hazard plotting for censored failure data. *Technometrics*, 14(4): 945–966, 1972. doi: 10.1080/00401706.1972.10488991. [p9]

J. Otto and D. Kahle. ggdensity: Improved bivariate density visualization in R. *The R Journal*, 15(2): 220–236, 2023. doi: 10.32614/RJ-2023-048. [p1, 14]

T. L. Pedersen. *patchwork: The Composer of Plots*, 2025. URL https://CRAN.R-project.org/package=patchwork. R package version 1.3.2. [p14]

R. Pruim, D. T. Kaplan, and N. J. Horton. The mosaic package: Helping students to "think with data" using R. *The R Journal*, 9(1):77–102, 2017. doi: 10.32614/RJ-2017-024. [p14]

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026. URL https://www.R-project.org/. [p1]

D. D. Sjoberg, M. Baillie, C. Fruechtenicht, S. Haesendonckx, and T. Treis. *ggsurvfit: Flexible Time-to-Event Figures*, 2025. URL https://CRAN.R-project.org/package=ggsurvfit. R package version 1.2.0. [p15]

D. Turner, D. Kahle, and R. X. Sturdivant. *ggvfields: Vector Field Visualizations with ggplot2*, 2026. URL https://CRAN.R-project.org/package=ggvfields. R package version 1.0.1. [p1, 14]

H. Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1): 3–28, 2010. doi: 10.1198/jcgs.2009.07098. [p1, 14]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2nd edition, 2016. ISBN 978-3-319-24277-4. URL https://ggplot2-book.org/. [p1, 14]

H. Wickham, P. Danenberg, G. Csárdi, and M. Eugster. *roxygen2: In-Line Documentation for R*, 2026. URL https://CRAN.R-project.org/package=roxygen2. R package version 8.0.0. [p13]

L. Wilkinson. *The Grammar of Graphics*. Springer-Verlag New York, 2nd edition, 2005. doi: 10.1007/0-387-28695-0. [p14]

Dusty Turner
United States Military Academy
West Point, NY
dusty.s.turner@gmail.com

David Kahle
Baylor University
Waco, TX
ORCiD: 0000-0002-9999-1558
david@kahle.io

Rodney X. Sturdivant
Baylor University
Waco, TX
rodney_sturdivant@baylor.edu

James Otto
Alcon Inc.
Fort Worth, TX
jamesotto852@gmail.com